Exercices en C – Pointeurs et allocation dynamique

Corrigé des exercices 1, 2 et 3.

Exercice 1 – Fonction de swap via pointeurs

Écrire un programme en C qui :

1. Demande à l'utilisateur de saisir deux entiers a et b.
2. Implémente une fonction :

```
void swap(int *x, int *y);
```

qui échange les valeurs pointées par x et y à l'aide d'une variable temporaire.

3. Dans le `main`, appelle `swap(&a, &b)` puis affiche les nouvelles valeurs de a et b.

Contraintes :

1. L'échange doit se faire exclusivement avec les pointeurs.
2. Aucun `return` d'une valeur (tout passe par les adresses).

Solution

Dans cet exercice les pointeurs permettent d'accéder directement aux variables a et b depuis la fonction swap. Sans cette technique, il serait impossible pour la fonction swap d'effectuer son traitement (elle ne peut pas retourner plus d'une valeur en résultat).

En Python on aurait pu utiliser un tuple comme résultat de la fonction (pour retourner les deux valeurs interverties).

```
#include <stdio.h>

// Il est important de déclarer la fonction avant la fonction main.
// int *x et int *y sont des paramètres du type "adresse d'une variable
↪  entière".
// L'objet qui permet de manipuler une adresse se nomme un pointeur.
void swap(int *x, int *y)
{
    // on stock la valeur actuellement pointée par y (sauvegarde)
    int temp = *y;

    // la valeur actuellement pointée par y prend la valeur pointée par x.
    ↪
```

```c
    *y = *x;

    // la valeur actuellement pointée par x prend la valeur pointée par
    ↪  temp (l'ancienne valeur pointée par y).
    *x = temp;
}

int main(int argc, char const *argv[])
{
    int a = 10;
    int b = 22;

    printf("Avant le swap : a = %d, b = %d. \n", a, b);

    // l'opérateur "&" permet de récupérer l'adresse d'une variable
    swap(&a, &b);

    printf("Après le swap : a = %d, b = %d. \n", a, b);

    return 0;
}
```

Exercice 2 – Somme et moyenne d'un tableau via pointeurs

Écrire un programme en C qui :

1. Déclare un tableau d'entiers de taille 5, par exemple :

```
int tab[5];
```

2. Demande à l'utilisateur de saisir les 5 valeurs du tableau.
3. Implémente une fonction :

```
void calculer_somme_moyenne(int *t, int taille, int *somme, float
↪  *moyenne);
```

qui :

- parcourt le tableau en utilisant un pointeur ou l'arithmétique de pointeurs,
- calcule la somme des éléments,
- calcule la moyenne (type `float`),
- renvoie les résultats via `somme` et `moyenne`.

4. Affiche la somme et la moyenne dans le `main`.

Contraintes :

- Ne pas utiliser la notation `t[i]` dans la fonction.

Solution

La fonction (procédure) `calculer_somme_moyenne` efféctue des opérations :

1. calcul de la somme
2. calcul de la moyenne.

Pour donner le résultat de ces deux opérations elle ne peut pas utiliser le mécanisme de retour classique (via `return`).

En effet, `return` ne permet pas de retourner plus d'une valeur. Il est alors nécessaire d'utiliser des pointeurs pour permettre à la fonction de "ranger" ses résultats dans l'espace mémoire de la fonction appelante (même problèmatique que la fonction swap de l'exercice 1).

```
#include <stdio.h>

// Il est important de déclarer la fonction avant la fonction main.
// taille_tab contient la taille du tableau (le nombre d'éléments).
// tab correspond au tableau (addresse de la première case)
```

```c
// somme et moyenne sont des les adresses vers des variables qui recevront
↪  respectivement la valeur de la somme et celle de la moyenne.
void calculer_somme_moyenne(int taille_tab, int tab[taille_tab], int
↪  *somme, float *moyenne)
{
    // on s'assure que les variables pointées ne contiennent pas déjà une
    ↪  valeur initiale.
    // (pour la somme, si on ne fait pas cette opération le résultat risque
    ↪  d'être faux.)
    *moyenne = 0;
    *somme = 0;

    // On itère sur le tableau pour faire la somme de ses éléments
    // size_t est le type utilisé pour les index.
    int *iter = tab;
    for (size_t i = 0; i < taille_tab; i++)
    {
        // il est important d'utiliser l'opérateur de déréférencement (*)
        ↪  pour modifier la valeur pointée et non pas l'adresse.
        *somme += *iter;

        // Incrémenter un pointeur permet de passer à "l'adresse suivante"
        ↪  c'est l'équivalent de "tab[i]"
        iter++;
    }

    // calcul de la moyenne, il est important de modifier le type de
    ↪  "taille_tab" (ou somme) pour avoir un résultat flottant (cast)
    *moyenne = (*somme) / (float)taille_tab;
}

int main(int argc, char const *argv[])
{
    int notes[] = {10, 12, 13, 14, 20, 15};
    int somme = 0;
    float moyenne = 0;

    // l'opérateur "&" permet de récupérer l'adresse d'une variable
    calculer_somme_moyenne(6, notes, &somme, &moyenne);

    printf("Somme = %d, moyenne = %.2f.\n", somme, moyenne);
}
```

Exercice 3 – Tableau de taille dynamique rempli par l'utilisateur

Écrire un programme en C qui :

1. Demande à l'utilisateur un entier n strictement positif : la taille du tableau.
2. Alloue dynamiquement un tableau de n entiers avec `malloc`.
3. Demande à l'utilisateur de saisir les n valeurs.
4. Affiche tous les éléments du tableau ainsi que la valeur minimale et la valeur maximale.
5. Libère la mémoire avant de quitter.

Contraintes :

- Vérifier que `malloc` ne renvoie pas `NULL`. En cas d'erreur, afficher un message et terminer proprement.

Fonction suggérée :

```
void trouver_min_max(int *t, int taille, int *min, int *max);
```

Solution

Dans cet exercice, l'utilisateur choisi le nombre d'éléments que doit contenir le tableau.

Deux solutions possibles :

- Pré-allouer un tableau d'une taille suffisamment grande pour espérer couvrir l'ensemble des besoins de l'utilisateur
- Utiliser l'allocation dynamique :
    1. demander à l'utilisateur la taille du tableau (dynamique = connu uniquement lors de l'exécution)
    2. créer l'espace mémoire sur le tas (heap) via malloc
    3. ne pas oublier de libérer avec free une fois le traitement libéré

Quand on utilise malloc / realloc / calloc l'emplacement mémoire utilisé par le programme est le tas. Cet emplacement est uniquement accessible en utilisant des pointeurs (malloc retourne l'adresse vers l'espace mémoire).

```
#include <stdio.h>
#include <limits.h>
#include <stdlib.h>

void trouver_min_max(const int *t, int taille, int *min, int *max)
{
    *min = INT_MAX;
```

```c
    *max = INT_MIN;

    for (size_t i = 0; i < taille; i++)
    {
        // malgré que t soit un pointeur il est possible d'utiliser la
        ↪  syntaxe d'indexation (offset d'adresse) "t[i]"
        if (t[i] < *min)
        {
            *min = t[i];
        }
        if (t[i] > *max)
        {
            *max = t[i];
        }
    }
}

int main(int argc, char const *argv[])
{
    int n = 0;

    // On récupère le nombre d'éléments que l'utilisateur souhaite
    ↪  renseigner
    printf("Combien d'éléments souhaitez-vous renseigner ? ");
    scanf("%d", &n);

    // malloc permet de demander au système d'allouer n éléments (ici des
    ↪  entiers)
    int *tab = malloc(n);

    // malloc retourne l'adresse vers l'emplacement mémoire alloué
    // dans le cas où malloc retourne NULL il s'agit d'une erreur (échec de
    ↪  l'allocation)
    if (tab == NULL)
    {
        printf("Une erreur d'allocation est survenue.");
        return 1;
    }

    // Remplissage du tableau
    for (size_t i = 0; i < n; i++)
    {
        printf("Saisissez la valeur N°%zu : ", i);
```

```
        scanf("%d", &tab[i]);
    }

    int min = 0;
    int max = 0;

    trouver_min_max(tab, n, &min, &max);

    printf("Minimum = %d, maximum = %d\n", min, max);

    // Quand on fait appel à l'allocation dynamique il est important de
    ↪  "nettoyer" (désallouer) la mémoire une fois qu'elle n'est plus
    ↪  utilisée
    free(tab);

    return 0;
}
```